

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 14
"SORTING"



DISUSUN OLEH:
Elsa Dwi Rizqiyanti
109082500090
S1 IF-12-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M. Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Bahasa pemrograman Go atau Golang merupakan bahasa pemrograman yang dikembangkan oleh Google dan dirancang untuk mendukung proses pengembangan perangkat lunak yang cepat, sederhana, serta efisien. Golang memiliki sintaks yang mudah dipahami dan mendukung pemrograman terstruktur sehingga cocok digunakan dalam pembelajaran algoritma dan struktur data. Dalam pemrograman Golang, array digunakan sebagai struktur data untuk menyimpan sekumpulan data dengan tipe yang sama dalam jumlah tetap. Array memudahkan proses pengolahan data seperti pencarian, pengurutan, dan manipulasi elemen data secara sistematis. Selain array, Golang juga mendukung penggunaan tipe data struct yang berfungsi untuk mengelompokkan beberapa variabel dengan tipe data berbeda ke dalam satu kesatuan data sehingga mempermudah pengelolaan informasi yang lebih kompleks, seperti data buku pada perpustakaan.

Salah satu konsep penting dalam pengolahan data adalah algoritma pengurutan (*sorting*). Pada praktikum ini digunakan metode *Insertion Sort*, yaitu algoritma pengurutan sederhana yang bekerja dengan cara membandingkan dan menyisipkan elemen pada posisi yang tepat sehingga data menjadi terurut. Metode ini cukup efektif digunakan pada jumlah data kecil hingga menengah karena memiliki implementasi yang mudah dipahami. Setelah data terurut, proses pencarian data dapat dilakukan lebih cepat menggunakan algoritma *Binary Search*. Algoritma ini bekerja dengan membagi data terurut menjadi dua bagian secara berulang hingga data yang dicari ditemukan atau dinyatakan tidak ada. Penggunaan *Binary Search* lebih efisien dibandingkan pencarian linear karena memiliki kompleksitas waktu yang lebih kecil.

Selain proses pengurutan dan pencarian, program pada praktikum ini juga menerapkan konsep analisis data menggunakan operasi perhitungan selisih antar elemen array. Program memeriksa apakah setiap data memiliki jarak atau selisih yang tetap setelah diurutkan. Konsep ini berkaitan dengan pola barisan aritmatika dalam matematika diskrit, yaitu suatu susunan bilangan yang memiliki beda tetap antar suku. Dengan menggabungkan konsep array, struct, sorting, searching, dan analisis pola data, program dapat digunakan untuk menyelesaikan berbagai permasalahan pengolahan data secara terstruktur dan efisien menggunakan bahasa pemrograman Golang.

SOAL LATIHAN SELECTION SORT

1. Soal 1

Source Code:

```
package main

import "fmt"

func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIndex := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIndex] {
                minIndex = j
            }
        }
        arr[i], arr[minIndex] = arr[minIndex], arr[i]
    }
}

func main() {
    var n int
    _, err := fmt.Scan(&n)
    if err != nil || n <= 0 || n >= 1000 {
        return
    }

    hasil := make([][]int, n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        daerah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&daerah[j])
        }

        selectionSort(daerah)

        hasil[i] = daerah
    }

    for i := 0; i < n; i++ {
        for j := 0; j < len(hasil[i]); j++ {
            if j > 0 {
                fmt.Print(" ")
            }
        }
    }
}
```

```

        fmt.Print(hasil[i][j])
    }
    fmt.Println()
}
}

```

Output:

```

18 func main() {
19     // ...
20 }
21
22 hasil := make([][]int, n)
23
24 for i := 0; i < n; i++ {
25     var m int
26     fmt.Scan(&m)
27
28     daerah := make([]int, m)
29     for j := 0; j < m; j++ {
30         fmt.Scan(&daerah[j])
31     }
32
33     selectionSort(daerah)
34
35     hasil[i] = daerah
36 }
37
38 for i := 0; i < n; i++ {
39     for j := 0; j < len(hasil[i]); j++ {
40         if j > 0 {
41             fmt.Print(" ")
42         }
43         fmt.Print(hasil[i][j])
44     }
45     fmt.Println()
46 }
47
48 }
49
50 }

```

Terminal Output:

```

PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 14> go run no1selection.go
3
5 2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 1
1 2 7 9 13
15 27 39 75 133 189
1 4 9

```

Deskripsi Program:

Program ini dibuat untuk mengolah dan mengurutkan data bilangan dari beberapa daerah menggunakan algoritma *Selection Sort*. Program diawali dengan membaca jumlah daerah yang akan diproses, kemudian untuk setiap daerah pengguna memasukkan jumlah data serta nilai-nilai bilangan yang akan disimpan ke dalam array. Setelah seluruh data dimasukkan, program melakukan proses pengurutan dari nilai terkecil ke terbesar menggunakan metode *Selection Sort*, yaitu dengan mencari elemen terkecil pada bagian array yang belum terurut lalu menukarkannya ke posisi yang sesuai. Hasil pengurutan dari setiap daerah kemudian disimpan dan ditampilkan kembali ke layar dalam keadaan terurut secara ascending. Program ini menunjukkan penerapan array dinamis (*slice*), perulangan, fungsi, serta algoritma pengurutan dalam bahasa pemrograman Golang.

2. Soal 2

Source Code:

```
package main

import "fmt"

func sortAsc(arr []int) {
    for i := 1; i < len(arr); i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }
}

func sortDesc(arr []int) {
    for i := 1; i < len(arr); i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] < key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }
}

func main() {
    var n, m, x int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var ganjil, genap []int

        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&x)

            if x%2 == 0 {
                genap = append(genap, x)
            }
        }
    }
}
```

```

    } else {
        ganjil = append(ganjil, x)
    }
}

sortAsc(ganjil)
sortDesc(genap)

for _, v := range ganjil {
    fmt.Print(v, " ")
}

for j, v := range genap {
    if j == len(genap)-1 {
        fmt.Print(v)
    } else {
        fmt.Print(v, " ")
    }
}

fmt.Println()
}
}

```

Output:

The screenshot shows a VS Code editor with a Go file named 'no2selection.go'. The code defines a 'main' function that takes an integer 'n' and a slice of integers 'x'. It iterates through 'x' and appends elements to 'ganjil' (odd numbers) or 'genap' (even numbers) based on whether the index is even or odd. After sorting 'ganjil' in ascending order and 'genap' in descending order, it prints both slices. The output in the terminal shows the sorted arrays: 5 2 1 7 9 13 and 6 189 15 27 39 75 133.

```

func main() {
    var n, m, x int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var ganjil, genap []int

        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&x)

            if x%2 == 0 {
                genap = append(genap, x)
            } else {
                ganjil = append(ganjil, x)
            }
        }

        sortAsc(ganjil)
        sortDesc(genap)

        for _, v := range ganjil {
            fmt.Print(v, " ")
        }

        for j, v := range genap {
            if j == len(genap)-1 {
                fmt.Print(v)
            } else {
                fmt.Print(v, " ")
            }
        }

        fmt.Println()
    }
}

```

Output:

```

5 2 1 7 9 13
6 189 15 27 39 75 133

```

Deskripsi Program:

Program ini dibuat untuk mengelompokkan dan mengurutkan nomor rumah kerabat berdasarkan aturan tertentu agar Hercules menyeberang jalan sesedikit mungkin. Program menerima masukan berupa beberapa daerah, kemudian pada setiap daerah pengguna memasukkan sejumlah nomor rumah. Nomor rumah ganjil dan genap dipisahkan ke dalam dua slice yang berbeda. Data nomor ganjil diurutkan secara menaik (*ascending*) menggunakan algoritma *Insertion Sort*, sedangkan nomor genap diurutkan secara menurun (*descending*). Setelah proses pengurutan selesai, program menampilkan seluruh nomor rumah ganjil terlebih dahulu dari kecil ke besar, kemudian dilanjutkan dengan nomor rumah genap dari besar ke kecil pada setiap daerah. Program ini menerapkan konsep array dinamis (*slice*), percabangan, perulangan, fungsi, serta algoritma pengurutan dalam bahasa pemrograman Golang.

3. Soal 3

Source Code:

```
package main

import "fmt"

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1
        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}

func hitungMedian(arr []int) int {
    n := len(arr)
    if n%2 != 0 {
        return arr[n/2]
    } else {
        tengah1 := arr[(n/2)-1]
        tengah2 := arr[n/2]
        return (tengah1 + tengah2) / 2
    }
}

func main() {
    var data []int
    var val int
```

```

for {
    _, err := fmt.Scan(&val)
    if err != nil {
        break
    }

    if val == -5313 {
        break
    }

    if val == 0 {
        if len(data) > 0 {
            insertionSort(data)
            median := hitungMedian(data)
            fmt.Println(median)
        }
    } else {
        data = append(data, val)
    }
}
}

```

Output:

The screenshot shows a Visual Studio Code editor with a Go file named `no3selection.go` open. The code implements an insertion sort and a median calculation function. The `main` function reads integers from standard input until it encounters `-5313` or `0`. When `0` is entered, it sorts the array and prints the median.

```

package main

import "fmt"

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1
        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}

func hitungMedian(arr []int) int {
    n := len(arr)
    if n%2 != 0 {
        return arr[n/2]
    } else {
        tengah1 := arr[(n/2)-1]
        tengah2 := arr[n/2]
        return (tengah1 + tengah2) / 2
    }
}

func main() {
    var data []int
    var val int

```

The terminal output shows the command `run no3selection.go` being executed, resulting in the output: `7 23 11 0 5 19 2 29 3 13 17 0 -5313`. A small window in the foreground displays the output of the program: `Elsa Dwi Rizqiyanti` and `1090821500090`.

Deskripsi Program:

Program ini dibuat untuk membaca sekumpulan bilangan bulat yang dimasukkan pengguna kemudian menentukan nilai median dari data tersebut. Data dimasukkan satu per satu dan akan disimpan ke dalam slice selama nilai yang dimasukkan bukan 0 atau -5313. Jika pengguna memasukkan angka 0, program akan mengurutkan seluruh data menggunakan algoritma *Insertion Sort* dari nilai terkecil ke terbesar, kemudian menghitung median data. Median diperoleh dari nilai tengah apabila jumlah data ganjil, atau rata-rata dua nilai tengah apabila jumlah data genap. Hasil median kemudian ditampilkan ke layar. Proses input akan terus berlangsung hingga pengguna memasukkan nilai -5313 sebagai penanda akhir program. Program ini menerapkan konsep slice, perulangan, pengurutan data, serta perhitungan statistik sederhana menggunakan bahasa pemrograman Golang.

SOAL LATIHAN **INSERTION SORT**

Soal 1

Source Code:

```
package main

import "fmt"

const NMAX = 100

func insertionSort(arr *[NMAX]int, n int) {
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > temp {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = temp
    }
}

func cekJarakTetap(arr [NMAX]int, n int) bool {
    if n <= 2 {
        return true
    }

    selisih := arr[1] - arr[0]

    for i := 2; i < n; i++ {
        if arr[i]-arr[i-1] != selisih {
            return false
        }
    }

    return true
}

func main() {
    var arr [NMAX]int
    var x, n int

    for {
        fmt.Scan(&x)

        if x < 0 {
            break
        }
    }
}
```

```

    }

    arr[n] = x
    n++
}

insertionSort(&arr, n)

for i := 0; i < n; i++ {
    fmt.Print(arr[i], " ")
}
fmt.Println()

if cekJarakTetap(arr, n) {
    fmt.Println("Data berjarak", arr[1]-arr[0])
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output:

The screenshot shows a VS Code editor with a Go file named `no1.go`. The code defines a function `cekJarakTetap` that checks if the difference between consecutive elements in a sorted array is constant. The `main` function reads input from the user, sorts it using insertion sort, and then checks the result using `cekJarakTetap`.

The terminal output shows the following commands and results:

```

PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 14> go run no1.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 14> go run no1.go
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 14>

```

A small window titled "Elsa Dwi R" is also visible, displaying the text "Elsa Dwi Rizqiyanti" and "1090821500090".

Deskripsi Program:

Program ini dibuat untuk membaca sejumlah bilangan bulat non-negatif yang dimasukkan pengguna hingga ditemukan bilangan negatif sebagai penanda akhir input. Semua bilangan non-negatif tersebut disimpan ke dalam array kemudian diurutkan menggunakan metode *insertion sort* dari nilai terkecil ke terbesar. Setelah proses pengurutan selesai, program menampilkan isi array yang telah terurut dan memeriksa apakah selisih antar elemen yang berurutan memiliki jarak yang sama atau tetap. Jika semua selisih antar data sama, maka program menampilkan pesan “Data berjarak x” sesuai nilai selisihnya, sedangkan jika tidak sama maka program menampilkan pesan “Data berjarak tidak tetap”.

Soal 2

Source Code:

```
package main

import "fmt"

const NMAX = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating    int
}

type DaftarBuku [NMAX]Buku

func inputBuku(A *DaftarBuku, n *int) {
    fmt.Scan(n)

    for i := 0; i < *n; i++ {
        fmt.Scan(
            &A[i].id,
            &A[i].judul,
            &A[i].penulis,
            &A[i].penerbit,
            &A[i].eksemplar,
            &A[i].tahun,
            &A[i].rating,
        )
    }
}

func sortBuku(A *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := A[i]
        j := i - 1

        for j >= 0 && A[j].rating < temp.rating {
```

```

        A[j+1] = A[j]
        j--
    }

    A[j+1] = temp
}
}

func main() {
    var A DaftarBuku
    var n, cari int

    inputBuku(&A, &n)

    sortBuku(&A, n)

    fmt.Println("Buku Terfavorit:")
    fmt.Println(A[0].judul, A[0].penulis, A[0].penerbit, A[0].tahun)

    fmt.Println("5 Buku Rating Tertinggi:")
    for i := 0; i < n && i < 5; i++ {
        fmt.Println(A[i].judul, "-", A[i].rating)
    }

    fmt.Scan(&cari)

    kiri, kanan := 0, n-1
    ketemu := false

    for kiri <= kanan {
        tengah := (kiri + kanan) / 2

        if A[tengah].rating == cari {
            fmt.Println("Data Buku:")
            fmt.Println(
                A[tengah].judul,
                A[tengah].penulis,
                A[tengah].penerbit,
                A[tengah].tahun,
                A[tengah].eksemplar,
                A[tengah].rating,
            )
            ketemu = true
            break
        } else if cari > A[tengah].rating {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }
}

```

```

    if !ketemu {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

```

Output:

```

PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 14> go run no2.go
3
B01 LaskarPelangi AndreaBuntara Bentang 10 2005 95
B02 Atomichabits JamesClear Gramedia 7 2019 90
B03 Filosofiterasi HenryTanampring Kompas 5 2018 85
Buku Terfavorit:
LaskarPelangi AndreaBuntara Bentang 2005
5 Buku Rating tertinggi:
LaskarPelangi - 95
Atomichabits - 90
Filosofiterasi - 85
90
Data Buku:
Atomichabits JamesClear Gramedia 2019 7 90
PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 14>

```

Deskripsi Program:

Program ini dibuat untuk mengelola data buku pada sebuah perpustakaan menggunakan array dan tipe data struct. Program diawali dengan membaca jumlah buku beserta data setiap buku yang terdiri dari id, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating buku. Setelah data dimasukkan, program mengurutkan seluruh data buku berdasarkan rating tertinggi ke terendah menggunakan metode *insertion sort*. Selanjutnya program menampilkan buku dengan rating tertinggi sebagai buku terfavorit dan menampilkan maksimal lima buku dengan rating tertinggi. Program juga menyediakan fitur pencarian buku berdasarkan rating tertentu menggunakan metode *binary search*. Jika data buku dengan rating yang dicari ditemukan, maka informasi buku akan ditampilkan, sedangkan jika tidak ditemukan program akan menampilkan pesan bahwa tidak ada buku dengan rating tersebut.

DAFTAR PUSTAKA

[Go Documentation](#). (2024). *The Go Programming Language Documentation*.

[GeeksforGeeks - Insertion Sort](#). (2023). *Insertion Sort Algorithm*.

[GeeksforGeeks - Binary Search](#). (2024). *Binary Search Algorithm*.

[Programiz Golang Tutorial](#). (2022). *Learn Go Programming*.

[W3Schools Go Tutorial](#). (2023). *Go Tutorial*.